electronics

MDPI

Article

A Low-Cost, Energy-Aware Exploration Framework for Autonomous Ground Vehicles in Hazardous Environments

Iosif Polenakis*, Marios N. Anagnostou , Ioannis Vlachos and Markos Avlonitis

Department of Informatics, Ionian University, 49100 Corfu, Greece; managnost@ionio.gr (M.N.A.); gvlachos@ionio.gr (I.V.); avlon@ionio.gr (M.A.)
* Correspondence: ipolenakis@ionio.gr

Abstract

Autonomous ground vehicles (AGVs) are of major importance in exploration missions since they perform difficult tasks in changing or harmful environments. Mapping and exploration is crucial in hazardous areas, or areas inaccessible to humans, demanding autonomous navigation. This paper proposes a lightweight, low-cost AGV platform, which will be used in resource-constrained situations and aimed at scenarios like exploration missions (e.g., cave interiors, biohazard environments, or fire-stricken buildings) where there are serious security threats to humans. The proposed system relies on simple ultrasonic sensors when navigating and applied traversal algorithms (e.g., BFS, DFS, or A^*) during path planning. Since on-board microcomputers have limited memory, the traversal data and direction decisions are stored in a file located on an SD card, which supports long-term, energy-saving navigation and risk-free backtracking. A fish-eye camera set on a servo motor captures three photos ordered from left to right and stores them on the SD card for further off-line processing, integrating each frame into a low-frame-rate video. Moreover, when the battery level falls below 50%, the exploration path does not extend further and the AGV returns to the base station, thus combining a secure backtracking procedure with energy-efficient decisions. The resultant platform is low-cost, modular, and efficient at augmenting; thus it is suitable for exploring missions with applications in search and rescue, educational robotics, and real-time applications in low-infrastructure environments.

Keywords: autonomous unmanned ground vehicle; low-cost exploration framework; battery-aware path planning; off-line image stitching; hazardous environment exploration; embedded systems; energy-efficient robotics

check for updates

Academic Editor: Ning Chen

Received: 19 July 2025
Revised: 10 September 2025
Accepted: 12 September 2025
Published: 16 September 2025

Citation: Polenakis, I.; Anagnostou, M. N.; Vlachos, I.; Avlonitis, M. A Low-Cost, Energy-Aware Exploration Framework for Autonomous Ground Vehicles in Hazardous Environments. *Electronics* **2025**, *14*, 3665. https://doi.org/10.3390/electronics14183665

Correction Statement: This article has been republished with a minor change. The change does not affect the scientific content of the article and further details are available within the backmatter of the website version of this article.

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Autonomous robots play a major role in engineering as they utilize automated systems to accomplish complex tasks when in an environment that is dynamic and/or unsafe. Autonomous navigation within buildings or in post-disaster or otherwise inaccessible-to-human areas requires strong control algorithms, real-time detection of the environment, and responsive behavior. This paper introduces the framework of an AGV to autonomously explore such areas and capture extensive visual and mapping data. The main features of the proposed framework are the integration of a traversal algorithm-based navigation approach, battery-aware planned exploration, robust image capture, and a modular software architecture that allows extensibility and hardware abstraction. One of the main aspects of the proposed approach is the low-cost deployment of the AGV intended to navigate indoors by using low-cost components and an embedded traversal algorithm to explore

an unknown hazardous area. The aim is to collect spatially structured images that can be sequentially applied in off-line video generation. This work presents the design and architecture of the framework of an autonomous ground vehicle (AGV) meant to explore inaccessible/unsafe indoor environments where the proposed AGV utilizes a low-cost Raspberry Pi Zero W micro-controller with an interface to multiple sensors and actuators that assist in real-time obstacle sensing, image acquisition, and decision-making. Traversal algorithms are the core of the exploration basis, providing a navigation tool to move directly into the unknown areas and react dynamically to the environmental effects on the ultrasonic sensors. Each step of the AGV navigation captures a structured triad of images by the servo-driven fish-eye camera module and retains the images sequentially on an on-board SD card. These groups of images are intended to be merged in threes (with respect to their title and time of capture) and assembled off-line into video files after the mission (i.e., when the AGV has exited from the hazardous area, returning to the starting point). Battery level monitoring guarantees efficient and safe operation since there is the initiation of a backtracking routine once the battery level falls below a particular value (e.g., <50%). For the assessment of the proposed approach, a simulation environment is developed to investigate the behavior and the dynamics of the AGV, which proves the potentials of the design. The applications that this system will be targeted to are search and rescue, robotic inspection, and cave interior mapping.

1.1. Related Work

The possibility of autonomously navigating in resource-limited environments has been met with great expectation and interest because of its urgent applications in the areas of search-and-rescue missions, post-disaster site surveys, and exploration operations associated with high-risk zones. The proposed system extends to many research areas, such as low-cost robotics, energy-efficient path planning, external state recording, and off-line visual reconstruction.

Similarly to the work proposed in this study, recent advancements in autonomous mobile robotics have focused significantly on energy efficiency, obstacle detection, and environment mapping. Maidana et al. [1] introduce an energy-aware path planning approach that integrates energy consumption directly into the planning process, achieving notable reductions in energy usage during autonomous navigation. Similarly, Rappaport [2] and Mei et al. [3] present energy-efficient exploration strategies that minimize travel distance and optimize battery usage through adaptive thresholds and orientation-based decision-making. On the sensing and perception front, Teixeira et al. [4] propose a low-cost 3D reconstruction system using RGB-D sensors for complex environments like caves, providing an accessible solution for spatial data capture. In terms of obstacle detection, Rejab and Abd-Al Hussain [5] design an intelligent mobile robot using three ultrasonic sensors and an Arduino-based micro-controller, enabling autonomous navigation in unstructured environments. Azeta et al. [6] further explore ultrasonic sensing for obstacle avoidance, demonstrating flexibility and robustness under various conditions with a minimal hardware set-up. Finally, Kerstens [7] proposes the eRTIS system, which introduces a fully embedded 3D sonar sensor capable of real-time spatial localization using ultrasound, which is particularly suited for robotic applications in harsh or low-visibility conditions.

Regarding low-cost AGVs, there have been many attempts to investigate the viability of low-cost, single-board computers to be used to deploy autonomous ground vehicles (AGVs). Meneses et al. [8] used Raspberry Pi to provide an optical-flow-based navigation implementation that provides simple obstacle avoidance within structured environments. In a related approach Khan et al. [9] low-cost autonomous robot prototype was developed using a Raspberry Pi, ultrasonic sensor, and camera module to perform object detection

and image processing for navigation. Other projects, like using ultrasonic and infrared sensors to navigate and identify the presence of obstacles, have made use of lightweight embedded software to make real-time decisions, including those of Ziouzios [10] and the NTNU Open Project [11]. These papers reassert the increasing feasibility of advocating the use of AGVs in low-cost scenarios but with reduced sensing modalities.

In energy-aware exploration and battery-limited path planning, energy-constrained path planning of AGVs has received increasing interest. Seewald et al. [12] extend ergodic search methods by incorporating battery constraints, enabling robots to balance energy consumption with coverage quality during exploration. Ben Naveed et al. [13] presented a strategy applied in filtering exploration pathways depending on the remaining amount of energy. In the same aspect, Zhang et al. [14] propose an energy-saving optimization and control (ESOC) method that improves the energy utilization efficiency of autonomous electric vehicles by mapping driving scene constraints into the vehicle dynamics and control domain. Experimental results show that the ESOC effectively optimizes the powertrain's operation point distribution and achieves lower energy consumption compared to state-of-the-art methods. Dovgan et al. [15] examined dynamic weighting on the basis of DWA to optimize the planning of motion so as to minimize energy consumption. El-Hussieny et al. [16] also optimized graph traversal algorithms by adding heuristic backtracking to avoid redundancy and save power.

With the ongoing logging and external state management, given the progress in autonomous control, persistence state logging remains poorly discussed in low-resource AGVs. Parasuraman [17] propose a sensor fusion technique based on a Modified Fuzzy Associative Memory to improve mobile robot navigation in complex environments. It is novel that our work proposes a new logging mechanism based on SD cards, by which the traversal graphs, decision points, and energy levels are continuously logged and referred to, to guide behavior during the runtime—a process that makes the observation process robust and memory-less. In the example of graph-based approaches to path planning, classical traversal (exploration) algorithms like Breadth-First Search (BFS), Depth-First Search (DFS), and A^* are still extensively used on AGVs, which would only require deterministic, discrete path planning. These algorithms are computationally low-intensity, simple to realize, and fit finely into resource-constrained embedded processes [18]. The results shown in El-Hussieny et al. [16] indicate that DFS with backtracking is capable of outperforming the probabilistic algorithms in structured settings with energy and computational constraints.

As to the logging and off-line visual reconstruction of the images captured by the camera, whereas most of the real-time vision-based systems assume high-performance compute units, off-line visual processing is a sufficient alternative. Image capture with time-stamps is typically used as the means of performing a 3D reconstruction after a mission, or stitching of a video in the UAV and underwater robot literature [19–21]. Panoramic image triads are not, however, the first example of structured capture around each decision point; here this process is facilitated on a servo-mounted fish-eye camera, which is unique in terrestrial AGVs. These triads in our method are sequentially stored on an SD card, which can latterly be reconstructed into low-frame-rate visual maps or videos of the search path (which are useful in post-mission analysis and situational awareness). Further, even regarding more generalized use and machine learning, in related fields, Meneses et al. [8] and Kim et al. [9] proposed vision-based and SLAM approaches on resource-constrained platforms, and Reyes et al. [22] explored the use of neural networks to predict energy requirements during exploration. In [23], Byun et al. develop an energy-efficient control method for robot swarm networking systems (RSNS). They design a self-triggered propagation model that allows system configurations to spread through the swarm via local rules, reducing the need for constant communication and energy saving.

Two quite notable works have been proposed in [24,25]. Recent advances in intelligent navigation control have explored AI-driven methods for trajectory planning and environment perception. For example, a Broad Learning (BL)-based controller has been proposed for AGVs, integrating optimal control problem (OCP) formulations with off-line training to achieve real-time trajectory planning within obstacle constraints [24]. This approach retains the optimality of traditional control methods while reducing the long off-line training times common to deep neural network (DNN) methods and supports local re-planning when the environment changes. Similarly, AI-based perception methods have been developed for object detection and distance estimation in unknown environments [25]. In this case, pretrained neural networks such as YOLO and MiDaS are applied to LiDAR-derived spatial data, combining vision-based detection with laser scanning for high-accuracy distance measurement. These methods provide precise scene understanding and dynamic obstacle handling but often require higher computational resources, sensor complexity, and power budgets. In comparison, our proposed system adopts a minimalist sensor suite (ultrasonic sensing and low-resolution imaging) and low-complexity traversal algorithms stored on an SD card to enable energy-efficient, long-duration navigation in resource-constrained environments. While BL- or deep-learning-based controllers can deliver higher adaptability and precision in well-resourced set-ups, our approach prioritizes modularity, low-cost components, and secure backtracking for applications in hazardous or infrastructure-poor conditions.

The approaches proposed in the literature point to the need to incorporate energy awareness into the fundamental navigation logic, something that is specifically dealt with within our system, as the backtracking capabilities and real-time battery monitor help us to guarantee its operating capability without GPS or SLAM. Overall, the existing literature has established the relevant foundations with regard to such isolated aspects of affordable autonomy, energy-aware planning, and off-line mapping. However, relatively little research has been conducted that integrates them into a consistent system of deployment. The framework proposed in this work (i) provides the utilization of known traversal algorithms to assist secure navigation, (ii) employs energy-aware decision-making, and (iii) writes structured image triads on an SD card to later reconstruct visual information in a post-mission fashion—filling important gaps within the present research of AGVs with resource limits.

1.2. Motivation and Contribution

Autonomous ground vehicles (AGVs) are important exploration and data gathering platforms in inaccessible hazardous areas such as buildings that have collapsed, cave interiors, or chemically polluted areas. Nevertheless, most AGV-based products use high-cost and sensor-dense systems with complex SLAM and costly video processing procedures and cannot be deployed in low-cost and restricted conditions. This paper relates to the above need through the design of a low-cost AGV platform that is used autonomously to navigate through hazardous areas. The system is constructed based on a Raspberry Pi Zero W single-board computer using low-power ultrasonic sensors to sense obstacles in real time and a classical approach to graph traversal algorithms of autonomous navigation and decision-making. More importantly, the on-board computing systems are relatively scarce, so there is very little computational complexity required by the resources of the AGV. The proposed system implements a new concept in visual documentation: it adopts structured recording at every exploration step; that is, a servo-actuated fish-eye camera records a triad of images at pre-selected angular offsets. These images are then sequentially saved to an SD card whose file names show the movement index of the vehicle. Instead of trying to generate the video in real time, or to perform scene reconstruction, thus consuming

more energy, the sets of images are processed later off-line to create a coherent video that visually reconstructs the environment traversed. This design option vastly simplifies hardware but still allows the system to have recordable visual data useful for post-mission visual analysis, mapping, or review by the operator.

The main contributions of the framework presented in this work are listed below:

- **Low-cost autonomous exploration platform:** A co-designed hardware–software AGV is used with components optimally selected in terms of price, modularity, and flexibility of deployment into constrained spaces.
- **Energy-aware navigation:** A battery management subsystem is included that actively manages behavior in response to the available energy, causing safe backtracking (i.e., return to starting point) before loss of energy.
- **State logging:** In addition, *SD card-based persistent state logging* allows the whole range of traversal decisions, energy states, and image triads to be stored on an SD card, which makes the implementation attractive for resource-constrained platforms by allowing robust and memory-efficient exploration.
- **Off-line video reconstruction:** A wide-angle camera placed on a servo mount captures ordered images (sets of three images per capturing) and the off-line video reconstruction utilizes these triads of sequential images at each decision point to later form a panoramic image. The captured triads of images are merged into single frames that are later ordered to form a video.

The proposed framework, incorporating path planning algorithms, energy-aware behavior, and off-line visual processing, provides a lightweight, low-cost, and energy-aware exploration approach that complements full SLAM. The modular architecture of the system can be easily modified in the future to incorporate advanced capabilities, such as visual SLAM, machine learning, computer vision, or coping with cloud computing needs, fitting to the requirements set by each case.

2. Materials and Methods

Next, we present the theoretical background comprising the basis of the proposed framework, including the routing algorithm utilized for the decision-making of the AGV during the exploration of an area, energy management, a comparative analysis of the investigated algorithm deployed for path exploration and backtracking, and how they are related to energy consumption, as well as the challenges posed during the exploration of unknown areas compared to the traversal of a graph whose structure is already known.

2.1. Investigation Algorithm

The algorithm behind decision-making during the exploration of an unknown area or the traversal of a graph that is a map (e.g., a maze) deploys the known procedures from BFS, DFS, or A^* with a root node (in our case, the starting point). The distinction that rises in the case of the exploration of an unknown area results from the fact that the topology of the surrounding area is not known at the beginning of the exploration, and thus the path planning algorithms need to be slightly modified so as to match the case where the end of the traversal is not known. Particularly, regarding, e.g., the BFS and DFS algorithms, when the traversal reaches a leaf vertex it automatically resumes the search from the pending vertex, while in our case the AGV needs to move through all the previous vertices to the pending vertex, thus consuming energy.

Each movement consumes a portion of the battery level based on the utilization of the peripheral sensors, while when the battery level reaches a level <50% the AGV reverses its direction and covers the way that it arrived to the current node through a *backtracking* procedure.

Based on the most important variables of the algorithm, except the variables n, m, which correspond to $V(G), E(G)$, respectively, for a topology considered as a graph G, note that the edges of the underlying structure of the surrounding area match the neighboring non-obstacle points located to the north, east, south, and west of the current point (i.e., points that correspond to the incident vertices of that edge). Next, variable d corresponds to the battery cost per step considering the portion of the battery of the AGV consumed per step with respect to the power consumption required for the functionality of particular peripherals. Finally, B corresponds to the starting battery capacity. The theoretical maximum number of steps the AGV can perform until all the battery is exhausted is given by the variable $k = \left\lfloor \frac{B}{d} \right\rfloor$, while in our case in order for the *backtracking* to be initiated it should hold that the battery level is lower than 50%.

Regarding the complexity analysis (as far as the time complexities and space complexities are considered) the following procedures are to be taken into account:

- **Search Initialization.** Data structures are established just before exploration begins, with such items as the visited set, movement logs and path logs, and the frontier (a queue or stack with BFS or DFS, respectively). Such operations require constant time, and they require a complexity of $O(1)$.
- **Exploration Loop.** The main exploration routine involves repeatedly consuming battery, taking one node on the frontier, examining its neighbors, and updating logs, while the procedure continues until the battery runs low/empty. As every single node is explored just once and moves are limited by k, a step will take place within $\mathcal{O}(k)$ time where $k \leq n$.
- **Backtracking on Low Battery.** As soon as the battery level of the AGV falls below 50%, it reverses its way back by recording every move and loses more battery with every step. This phase is also $\mathcal{O}(k)$, because as the path has already been stored, the backtracking on it will occupy not more than k steps. On this point we should note that since the path so far is recorded alongside the decision point, through the *backtracking* procedure the reversed path is followed without requiring the peripherals to be utilized, thus saving battery level as it consumes less energy.
- **Total Time and Space Complexity.** As exploration and backtracking have time complexity up to $\mathcal{O}(k)$ each and $k \leq n$, the total time complexity is $\mathcal{O}(n)$. The level of space complexity is also $\mathcal{O}(n)$ as a result of storing of the visited set, movement records, and the path history.

The featured exploration algorithm will allow the AGV to execute energy-based traversal of the graph. It requires two logs, including *MovementLog*, which stores the history of movements and energy-related incidents, and *PathLog*, which stores the nodes visited as well as the remaining battery percentage. A queue (in the case of BFS) or a stack (in the case of DFS) is (initially) created as a frontier of exploration depending on the chosen type of algorithm. During traversal, nodes are visited and marked, and each move drains a fixed amount of battery power. In the case that the battery level reaches a critical point (<50%), the AGV enters a backtracking operation where it will reverse the path previously logged, returning step by step to the base, updating both logs. This arrangement makes sure that there is a safe return to the starting point (i.e., the *backtracking* procedure is enabled) at a time the power will not be fully exhausted. In the case that the battery hits zero at any given step as it explores forwards or when backtracking, it terminates the process and records the failure in the battery. At every point in the search, neighbouring nodes are retrieved and points that are not visited are put into the frontier in order to traverse them further. Persistent traversal, logging, and energy monitoring enable the AGV to achieve robust, energy-efficient, memory-efficient exploration with fail-safe return-to-base capability in the face of energy constraints. The step-by-step procedure is explained in Algorithm 1.

Algorithm 1: Explore procedure (AGV map search with backtracking).

Input: Graph G, Start node $start$, Algorithm type $algo$, Battery B, Drain per step d
Output: MovementLog, PathLog

begin
  $Visited \leftarrow \emptyset$
  $MovementLog \leftarrow$ empty list
  $PathLog \leftarrow$ empty list
  $B_{left} \leftarrow B$
  if $algo = BFS$ **then**
    $Frontier \leftarrow queuewith(start, [])$
  else
    $Frontier \leftarrow stackwith(start, [])$
  while *Frontier not empty* **do**
    if $algo = BFS$ **then**
      $(current, path) \leftarrow Frontier.popFront()$
    else
      $(current, path) \leftarrow Frontier.pop()$
    if $current \in Visited$ **then**
      continue
    $Visited \leftarrow Visited \cup \{current\}$
    $B_{left} \leftarrow B_{left} - d$
    if $B_{left} \leq 0$ **then**
      $MovementLog.append(\text{“}Battery\ depleted!\text{”})$
      break
    if $path \neq \emptyset$ **then**
      $(dir, _) \leftarrow path[-1]$
      $MovementLog.append(\text{“}dir\ to\ current\text{”})$
    $PathLog.append((current, B_{left}))$
    if $B_{left} < 50$ **then**
      $MovementLog.append(\text{“}Battery\ low\ (B_{left}\%) - returning\ to\ base...\text{”})$
      $BacktrackPath \leftarrow reverse(path)$
      foreach $(dir, prev) \in BacktrackPath$ **do**
        $revDir \leftarrow$ `ReverseDirection`(dir)
        $B_{left} \leftarrow B_{left} - d$
        if $B_{left} \leq 0$ **then**
          $MovementLog.append(\text{“}Battery\ depleted\ while\ backtracking!\text{”})$
          break
        $MovementLog.append(\text{“}revDir\ to\ prev\text{”})$
        $PathLog.append((prev, B_{left}))$
      break
    $neighbors \leftarrow$ `GetNeighbors`$(G[current])$
    foreach $(dir, neighbor) \in neighbors$ **do**
      if $neighbor \notin Visited$ **then**
        $Frontier.append((neighbor, path + [(dir, neighbor)]))$
  return $(MovementLog, PathLog)$

The procedures *GetNeighbors()* and *ReverseDirection()* assist in the primary exploration algorithms and help in traversing and navigation logic on the graph. The *GetNeighbors()* procedure pulls a legitimate directional association out of the data structure of the given node, which parallels methodologies for finding expected neighbouring nodes based on directions. The procedure iterates over each entry and non-null or undefined destinations are removed; then a list of valid pairs $(Direction, Destination)$ is returned. This is to make sure that path planning only considers those potential options of movement. The *ReverseDirection()* method returns the logical opposite of an entered direction command, the counter-direction (e.g., north returns south). It is necessary to apply the backtracking behavior so that in the case of a movement to a previous location the AGV is able to go back to the same place. Altogether these processes help to make the exploration framework modular and transparent, encouraging sustainability and proper directional movement within the limited situations. Step-by-step procedures are outlined in Algorithms 2 and 3, respectively.

Algorithm 2: GetNeighbors.

```
Input: NodeData: map of directions to nodes
Output: List of (Direction, Destination)
begin
    neighbors ← empty list
    foreach (dir, dest) ∈ NodeData do
        if dest ≠ None then
            neighbors.append((dir, dest))
    return neighbors
```

Algorithm 3: ReverseDirection.

```
Input: Direction d
Output: Opposite Direction
begin
    return  if d = "North" then
        "South"
    else if d = "South" then
        "North"
    else if d = "East" then
        "West"
    else if d = "West" then
        "East"
```

2.2. Energy Management

The logic behind the autonomous navigation of the AGV inside an unknown area is affected by the energy consumption during traversal. The AGV explores a 2D area using a traversal algorithm (e.g., BFS, DFS, or A^*). Each point in the area is either free (navigable) or blocked (obstacle). The AGV begins exploration from a starting point and proceeds to visit all reachable points until one of the following conditions is met:

- All navigable points have been visited (full exploration).
- The remaining battery energy is below the critical threshold of 50%, initiating an emergency return to start (i.e., *backtracking* procedure).

At each step, the AGV selects a target point following the current exploration strategy. When the next target point is not adjacent, the AGV will next estimate a path to its next target point based on the searching algorithm over the already visited points. The adherence to the path is in turn followed step by step. Every move, the planning of paths, and backtracking are noted down in time-stamped records.

To start with, the total energy budget of the AGV is set to be initially equal to B units. The operative functions of the AGV define the energy that is utilized in each step of the movement:

- **Base Movement Cost:** Each step forward, regardless of direction, costs 1 unit of energy.
- **Camera Operation Cost:** The AGV captures 3 images per step for visual inspection and mapping, resulting in a fixed additional cost of 3 units per step.
- **Sonar Directional Cost:** A sideways sonar scan is triggered when the AGV changes movement direction (e.g., from north to east). Each directional change incurs an extra cost of 2 units.

Let n be the total number of movement steps and k be the number of times the AGV changes its movement direction during the traversal. The total energy consumption E_{total} of the AGV during maze traversal is given by:

$$E_{\text{total}} = n \cdot (1 + 3) + k \cdot 2 = 4n + 2k \tag{1}$$

The provided computation accounts for the *base movement cost* of 1 unit per step, the *servo camera cost* of 3 units per step (3 images captured), and the *sideways sonar cost* of 2 units, applied only when the direction changes. For an initial energy level of, e.g., 1000 units, the remaining energy at any point in time is therefore:

$$E_{\text{remaining}} = E_0 - E_{\text{total}} = 1000 - (4n + 2k) \tag{2}$$

To prevent complete depletion, the AGV continuously checks whether the remaining energy is less than $\frac{E_0}{\texttt{battery_low}} = \frac{1000}{2} = 500$ units. If so, the AGV stops exploring and begins a return path to the initial start position utilizing the path of the tree followed from the starting point to the point where the energy level falls below 50%, moving across the already visited cells. The model prioritizes efficient exploration while balancing energy consumption through adaptive return strategies and algorithmic path planning.

2.3. Traversal Constraints in Unknown Maze Exploration

As used in classical graph theory, the traversal algorithms have a standard computational complexity on completely known graphs. This theoretical parity, however, no longer holds in situations where the environment is unknown and has to be discovered over time, such as in AGV exploration. The variations in operational efficiency and utilization of resources is accentuated by the costs of movement as well as the physical constraints of the robot.

The high efficiency derived from BFS and DFS supposes that the topology of the graph and its nodes and connections are entirely known before traversing. The main difference in them is their exploration behaviors: DFS searches as far as it can before going back to a previous state, whereas BFS searches through all neighbors at the current depth and then moves to the next step. In these circumstances, both algorithms perform with similar computation requirements and will ensure reliable, complete exploration.

In practical situations like the navigation of a robot in unknown territory, such an equivalence can no longer be assumed. As with the case of an Autonomous Guided Vehicle (AGV) exploring a maze, individual nodes need to be found gradually as one

would not have instant and free access to them any more. Rather than being exploratory, real-time environment sensing and locomotion are demanding in terms of energy and time. Because the AGV is required to build a model of the environment as it negotiates its way through it, both the algorithms used and the sheer physical geometry become limiting factors in traversal.

The deployment of BFS explores all neighbors at the current depth before moving to the next level. However, in an unknown maze, BFS requires the AGV to backtrack frequently to visit nodes at the frontier, even if they are physically distant. The robot incurs a high energy cost due to constant movement between already-visited areas and new frontiers, which can be scattered across the maze. Though BFS finds optimal paths between nodes in terms of steps, it is inefficient for full exploration in unknown terrain where parallel frontier processing is impossible (e.g., a single AGV cannot fan out simultaneously in multiple directions). In this context, traversal strategies work much differently. BFS, although uniform in familiar graphs, will be inefficient in unknown spaces because its frontier expansion policy is inefficient in unknown spaces. This means that the AGV is often expected to move between adverse sections of the environment that are spatially remote as far as a given depth is concerned. This creates a high degree of movement overhead, since the vehicle may have to backtrack across the environment several times in search of frontier nodes. The cost of energy per movement—as well as frequent sensor triggering—unnecessarily increases the use of operating costs. In unknown terrain BFS will likely result in inefficient and fragmented movement patterns even though it guarantees the discovery of the shortest paths in fully known graphs.

The deployment of DFS proceeds by exploring as far as possible along one path before backtracking. In the context of AGV exploration, DFS minimizes redundant movement. The robot continues in a single direction until it hits a dead end and then backtracks only once per branch. This behavior drastically reduces the number of direction changes and avoids excessive use of energy-intensive operations such as servo rotations and image capturing. Moreover while DFS does not guarantee the shortest path between nodes, its spatial and temporal locality make it highly energy efficient in unknown space traversal. DFS can be beneficial with respect to these limitations. The AGV moves along just a single, unbroken arm of environment, retracing its steps only in the case where a dead end is reached. Such linear sequencing reduces the number of long-distance moves and makes directional change minimal, leading to a drastic saving in the amount of energy expended. Exploration is carried out in a more focused and progressive way; thus the sensor work is also less frequent and more localized. Although DFS is not optimal in finding the most efficient path to each node, the orientation performance in physical exploration is closer to real-world unknown environments and thus leads to greater practical efficiency.

Finally, the deployment of the A^* algorithm enhances BFS by integrating a heuristic function, usually estimating the cost to reach a goal. However, in the AGV settings there is no single goal, and the AGV must explore all traversable areas, making the heuristic function (e.g., Manhattan distance to the start) largely irrelevant. A^* degenerates into behavior very similar to BFS under these conditions, as it cannot leverage its heuristic effectively to reduce exploration overhead. Consequently, A^* exhibits energy usage and traversal behavior almost indistinguishable from BFS. The A^* algorithm, which is highly accurate in standard path-finding because of its ability to make use of heuristics, loses its effectiveness when applied to situations of exploring rather than getting to a preset ending point. Such heuristic guidance is irrelevant in circumstances where the overall coverage of a non-localized goal is necessary without prior knowledge of a goal location. Consequently, A^* is similar to BFS in the sense that it also then places priority on nodes in terms of their vicinity and not their exploration value. Thus, the AGV also has to deal

with movement and sensor overhead as it brings itself to widely deployed frontiers. In the above exploration scenario, without any fixed objective to focus the heuristic, there is not much difference between A^* and BFS in practice.

As we can observe from Table 1 (pros and cons are denoted with ✓and ✗, respectively), DFS works well when exploring unknown and resource-constrained environments and when backtracking and sensor activation (e.g., servo-based image capture) are very costly. BFS and A^* may be good algorithmically in goal-directed search within known areas, but because they have no parallel movement abilities and often have to resort to repeat return-to-frontier operations, they are impractical in full exploration. In conclusion, all three algorithms, BFS, DFS, and A^*, have similar theoretical complexities, whereas these complexities do not work well in the real-world scenario as the performance in unknown settings differs considerably. The best solution, restricting long-distance movements and emphasizing path continuity, is a more energy-efficient and operationally feasible method to solve the problem that can be proposed by BFS. Both BFS and A^* are ineffective in physically exploring environments and can be costly in terms of energy consumption and poor coverage solutions.

Table 1. Comparison of exploration algorithms in unknown maze environments.

Algorithm	Complexity	Cost	Quality	Efficiency
DFS	$O(V+E)$	✓ Minimal	✗ Often sub-optimal	✓ Most efficient
BFS	$O(V+E)$	✗ High	✓ Optimal	✗ Least efficient
A^*	$O(b^d)$	✗ High	✓ Optimal (goal)	✗ Nearly same as BFS

3. System Architecture

Next, we discuss the components that comprise the architecture of the proposed framework as well as the core subsystems that cooperate to provide the main functionalities of the AGV during the exploration of an unknown environment.

3.1. System Components

The proposed framework is based on a modular architecture and operates at low costs, deploying an energy-efficient system in potentially dangerous or infrastructure-poor cases. The system consists of seven key subsystems that will form the capability of sensing, decision-making, actuating, and enabling persistent data management. The hardware and software systems are tightly coupled to support robust autonomous behavior. The following components constitute the physical system.

- **Microcomputer Board:** The microcomputer board is composed of the Raspberry Pi Zero board, which coordinates the whole system, aligning the operation of the sensors with the navigation functionalities and logging. This component acts as the central controller coordinating navigation, perception, and decision-making under limited memory and processing resources; it stores decisions and traversal data on an SD card for energy-efficient, long-term operation and reliable backtracking.
- **Navigation Sensors:** These consist primarily of ultrasonic sensors that guide the AGV through obstacle detection and simple environmental perception, ensuring low-cost yet effective autonomous navigation in resource-constrained settings.
- **Visual Mapping:** This employs a fish-eye camera mounted on a servo motor to capture sequential images (left, center, and right), stored for off-line integration into a low-frame-rate video, enhancing environmental awareness without requiring heavy on-board processing.

- **DC Motor System:** This provides mobility for the AGV in hazardous or inaccessible environments, executing control signals from the microcomputer to perform exploration and secure return-to-base operations when energy thresholds are reached.
- **Path Planning:** This implements traversal algorithms (BFS, DFS, and A^*) to guide exploration, balance efficiency and energy use, and ensure robust backtracking when retreat is required due to environmental hazards or energy constraints.
- **Energy Monitoring:** This tracks power consumption to enforce energy-aware decisions, halting exploration and initiating safe return when the battery level falls below 50%, ensuring mission continuity while protecting system integrity.
- **Logging Module:** This records traversal paths, sensor data, and decision history on the SD card, enabling reproducibility, secure backtracking, and post-mission analysis to validate performance and improve robustness.

In Figure 1 the corresponding procedures and the underlying subsystems and their relations are presented.

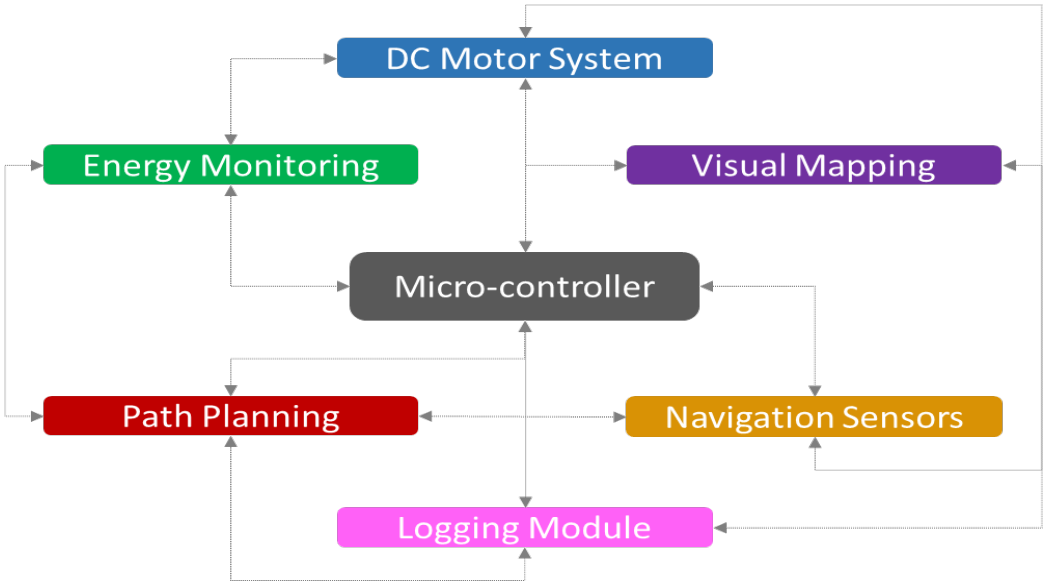


Figure 1. Framework architecture.

3.2. Navigation Subsystem

The essence of the system comprises the Raspberry Pi Zero, a small and low-energy microcomputer, which is used due to its very low power consumption, ease of programming, and compatibility with a very large variety of sensor modules. It is a central processing unit and executes algorithms of autonomous navigation, sensor input processing, power metrics, and motor control. The possibilities of using Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms with the path planning fits all the requirements of a limited memory footprint and deterministic nature of the chosen platform.

Raspberry Pi Zero W conducts all the computational routines, i.e., sensor data combination, management of motors, and initiation of image processing. Its small size, combined with the wireless connectivity built into it, makes it especially suitable to embedded robotic use. The power is provided by dual 18650 lithium-ion battery pack, which drives the motors and control electronics. There is an available power distribution module to control voltage regulation and circuit protection.

The motion subsystem consists of two L298N motor driver modules, which are able to handle four DC motors designed in a differential drive configuration. The two-axis workaround allows proper directional control, such as in-place rotations. For environmental awareness three ultrasonic sensors, i.e., HC-SR04, are placed at the front, left, and right

sides of the vehicle to detect the obstacles and distance measurements. Such readings are a part of a real-time navigation decision. At every crossroad the autonomous ground vehicle (AGV) considers the position of the forward, left, and right roads. Given a valid forward path, it will compute the necessary time to move a specified distance through the use of a known measured distance and a known velocity and move accordingly. The vehicle stores a group of three images after every forward movement with a type of camera that is servo-driven.

Four DC motors that are individually controlled enable the mobility of the AGV, which is facilitated by two H-bridge drivers (e.g., L298N) to change positions accurately with the capacity to steer and change with direction. The speed and direction of the motor are controlled in order to facilitate the decision-making of the path based on lateral maneuvers and controlled reversals of the backtracking operation. The vehicle also has ultrasonic and sonar sensors on the vehicle chassis to assist it in navigation. These sensors are based on time-of-flight techniques that help detect an obstacle and recognize a direction that is safe to cross in the immediate environment. Through constant sampling and processing of this data, the system builds a rudimentary local occupancy map, allowing obstacle avoidance and path optimization. The sensing method is a low-cost alternative to LiDAR systems and is quite insensitive to such adverse scenarios as low visibility or dust-heavy environments, e.g., a cave or the area after a disaster.

Regarding the topological map built using direction-based node transitions, the approximate inter-node distance could be computed without the aid of any other sensors, since we know that the standard velocity generated by the motors remains the same and we also know that the fixed time gap between any two successive sensors gives a ratio such that the distance between any two adjacent nodes could be estimated. As the proposed approach causes the AGV to follow an almost fixed velocity during traversal, the displacement within a segment can be approximated by the product of velocity and time until next use of the sensor (e.g., wall detection/decision point). This technique offers an imprecise but invariant recovery of the relative edge length that can be coded directly into the topological map. Even though it does not provide a fine-grained variation regarding wheel slip or differentiated surface, the method provides a cost-effective and de-coupled way of augmenting the graph representation with distance approximation. Finally, the data logging structure may include movement time-stamps, which can be used for approximate distance inference. Such methods, with periodic calibration (e.g., by comparing expected and measured arrival times), can be used to provide sufficiently precise estimates to support loop closure detection, path costweighting, and global optimization, furthering its usefulness to the map without contradicting the system philosophy of low cost and minimalism.

Finally, to improve robustness the proposed approach may host basic fault handling and monitoring features. These include timeout monitoring for sonar readings, logging of invalid sensor values or stalled movement events, and runtime checks for divergence from expected paths (e.g., detecting persistent obstacles or failed rotations). Additionally, upon detecting a sensor fault or navigation error, the AGV automatically initiates backtracking from its current position, regardless of the remaining battery level, as we will discuss later under Section 3.4. These simple diagnostics ensure safe return to the start point and allow continued evaluation of energy-aware exploration and traversal algorithm performance

Heading Error Compensation

Heading error remains a critical challenge in resource-constrained exploration, as rotational drift accumulates over time and degrades path accuracy. To mitigate this, lightweight inertial measurement units (IMUs) can fuse angular velocity and odometry for drift cor-

rection, but they introduce added cost, power, and software complexity. A cost-efficient alternative is a single gyroscope, which provides direct angular velocity for heading estimation. Although integration drift and bias instability limit gyro-only use, these issues can be alleviated through lightweight correction heuristics that exploit environmental features and operational redundancy. In this configuration, heading is estimated from yaw rate integration while drift is reduced through opportunistic bias adaptation, snapping to canonical headings after turns, and re-anchoring via ultrasonic wall and corridor alignment. While less robust than a full IMU, which typically combines a gyroscope, accelerometer, and magnetometer, a gyro-only set-up offers a lower cost, minimal complexity, and sufficient accuracy for structured indoor environments. To further enhance stability under these constraints, we introduce eight heuristic rules (H1–H8) that complement gyroscope integration with ultrasonic feedback and simple calibration, thereby reducing cumulative drift and suppressing noise without the overhead of full sensor fusion frameworks.

- **H1 (Gyro Bias Leak):** This mechanism continuously estimates and compensates for gyroscope bias during straight-line motion, when the commanded angular velocity is near zero. By comparing measured and commanded yaw rates, the bias estimate is updated incrementally, limiting long-term drift accumulation.
- **H2 (Snap-to-Turn Correction):** When discrete turns (e.g., at corridor junctions) are completed, the heading estimate is "snapped" to the nearest canonical orientation (such as multiples of 90°). This heuristic reduces residual angular error after large maneuvers and exploits the structural regularity of indoor environments.
- **H3 (Magnetometer Complementary Correction):** If a magnetometer is available, its absolute heading estimate is blended into the gyroscope-based estimate using a complementary filter. This provides coarse global orientation anchoring, mitigating long-term drift at the cost of potential sensitivity to local magnetic disturbances.
- **H4 (Wall-Normal Alignment):** When front-facing ultrasonic pairs detect a wall, the angular offset relative to the wall normal is estimated geometrically. Small corrections are applied to realign the robot, improving navigation accuracy in bounded spaces such as hallways.
- **H5 (Corridor Alignment):** Similarly, side-facing ultrasonic pairs allow estimation of lateral symmetry within corridors. By correcting deviations when left and right ranges are imbalanced, the robot maintains central alignment, which reduces systematic angular drift during straight-line exploration.
- **H6 (Motion Consistency Check):** The predicted change in front range, given the forward velocity and prior distance, is compared against measured ultrasonic readings. Consistent discrepancies are attributed to angular drift and used for correction. This opportunistically leverages redundancy between odometry and range sensing for self-calibration.
- **H7 (Zero-Rate Window Calibration):** When the robot is stationary, both commanded and measured angular velocities should ideally be zero. By averaging residual gyro outputs during such intervals, bias estimates are updated with high confidence, reducing long-term systematic error.
- **H8 (Deadband and Hysteresis):** To avoid excessive corrections from sensor noise, heading updates are applied only when deviations exceed a small threshold. Within this deadband, heading is smoothed using a hysteresis factor, preventing oscillations and overcompensation.

These heuristics provide complementary layers of error suppression. H1, H2, H7, and H8 primarily address gyro drift and calibration stability, while H3, H4, H5, and H6 exploit environmental structure and sensor redundancy for opportunistic correction. Importantly, all heuristics are designed to be computationally inexpensive and suitable for

low-power micro-controllers, aligning with the system's low-cost and energy-aware design philosophy.

In Algorithm 4 there are presented the steps followed during heading compensation. As we discussed previously, a single gyroscope (typically a MEMS angular rate sensor) can provide low-cost, low-power heading rate measurements suitable for short-term heading estimation. In particular, Algorithm 4 integrates gyroscope measurements with heuristic adjustments derived from ultrasonic sensing to achieve a lightweight yet robust heading estimation. At each iteration, the current heading estimate is initialized with the previously corrected value, after which the gyroscope bias is updated whenever the robot moves in a straight line, thereby reducing drift during long forward traversals. When a turning maneuver is completed, residual angular errors are suppressed by snapping the estimate to the nearest quantized orientation, while, if available, magnetometer readings are blended through a complementary update to anchor the heading globally. The core of the update remains gyroscope integration, corrected for estimated bias, which is then refined through environmental feedback: the difference between front-left and front-right ultrasonic sensors provides wall-normal alignment, whereas the disparity between side sensors supports corridor alignment. Additional consistency is enforced by comparing predicted forward distances with front sonar measurements, mapping residual discrepancies to yaw corrections. In stationary conditions with negligible rotation, the residual gyroscope output is attributed to bias and recalibrated, and, finally, small deviations are filtered through a deadband and hysteresis mechanism to avoid oscillations due to noise. Together, these heuristics operate synergistically, with some primarily addressing long-term drift while others exploit environmental geometry for situational corrections, yielding a stable and energy-efficient heading estimate suitable for low-cost robotic exploration.

Replacing an IMU with a gyro-only configuration reduces hardware complexity and cost while still allowing useful heading correction when combined with the lightweight heuristics and ultrasonic feedback described in Algorithm 4. However, gyro-only solutions require explicit handling of bias and integration drift, and they lack independent translational motion sensing (accelerometers) and absolute heading references (magnetometers).

3.3. Logging Subsystem

A wide-angle fish-eye is installed and a camera is mounted on a servo motor to make panoramic images. Each step of forward navigation is processed by the camera, which takes three images at discrete angles, so the overall area of the environment can be documented completely in space. Such images help to add a visual context that can be complementary to the sensor-based navigation and, in particular, mapping and retroanalysis. The information provided by the sensors is constantly checked to figure out whether a forward, left, or right movement is viable. When there is a barrier in every direction, the system performs the backtracking routine.

Captured pictures can be stored on a locally connected micro-SD card module that allows off-line data logging. The pictures are organized in packs of three pictures to make one iteration through the movement cycle, and the files are automatically numbered by the exploration stage (e.g., im_1.jpg, im_2.jpg, and im_3.jpg) merging them to a panoramic one as shown in Figure 2. The servo motor repositions the camera in a stepper fashion to the three predetermined angles at each capture cycle, providing a consistent panoramic record. A log of the path down which the robot passes is created in the form of a visual representation that allows one to analyze and reconstruct after a mission.

Algorithm 4: Angular correction.

Input: $\dot{\psi}_g$ (gyro yaw rate), ω_{cmd} (commanded yaw rate), v (forward speed), d_{FL}, d_{FR} (front-left/right sonar), d_L, d_R (side sonar), d_F (front sonar), L (sensor baseline)

Input: Parameters: $\alpha, k_w, k_p, k_c, \gamma, \beta, \tau, \psi_{thresh}, \lambda$

Output: Corrected heading ψ, updated bias $\hat{b}$

begin

$\psi^- \leftarrow \psi$

`// H1: Gyro Bias Leak (straight-line drift clamp)`

if $|\omega_{cmd}| < \omega_{thresh}$ ***and*** $v > v_{thresh}$ **then**

$\hat{b} \leftarrow \hat{b} + \alpha \cdot (\dot{\psi}_g - \omega_{cmd})$

`// H2: Snap-to-Turn Correction`

if *turning mode ended* ***and*** *robot settled* **then**

$\psi^- \leftarrow$ nearest quantized heading (e.g., 90° multiples)

`// H3: Magnetometer Complementary Correction (if available)`

if *magnetometer valid* **then**

$\psi_m \leftarrow \arctan 2(m_y, m_x)$

$\psi^- \leftarrow (1-\beta)\psi^- + \beta \cdot \psi_m$

`// Step: Gyro Integration`

$\psi^- \leftarrow \psi^- + (\dot{\psi}_g - \hat{b}) \cdot \Delta t$

`// H4: Wall-Normal Alignment (front ultrasonics)`

if d_{FL}, d_{FR} *valid* **then**

$e_\psi \leftarrow \arctan\big((d_{FR} - d_{FL})/L\big)$

if $|e_\psi| < \tau$ **then**

$\psi^- \leftarrow \psi^- - k_w \cdot e_\psi$

`// H5: Corridor Alignment (side ultrasonics)`

if d_L, d_R *valid* **then**

$e_\psi \leftarrow d_R - d_L$

if $|e_\psi| < \tau$ **then**

$\psi^- \leftarrow \psi^- - k_p \cdot e_\psi$

`// H6: Motion-Consistency Check (front sonar)`

if d_F *valid* ***and*** $v > v_{thresh}$ **then**

$d_{pred} \leftarrow d_F^{prev} - v \cdot \Delta t$

$e \leftarrow d_F - d_{pred}$

if $|e| < \tau$ **then**

$\psi^- \leftarrow \psi^- - k_c \cdot f(e)$ `// f(e) maps range residual to yaw error`

`// H7: Zero-Rate Window Calibration`

if $v < v_{stop}$ ***and*** $|\omega_{cmd}| < \omega_{stop}$ ***and*** $|\dot{\psi}_g| < \omega_{small}$ **then**

$\hat{b} \leftarrow (1-\gamma)\hat{b} + \gamma \cdot \dot{\psi}_g$

`// H8: Deadband and Hysteresis`

if $|\psi^- - \psi| < \psi_{thresh}$ **then**

$\psi^- \leftarrow (1-\lambda)\psi^- + \lambda \cdot \psi$

`// Final update`

$\psi \leftarrow \psi^-$

return $(\psi, \hat{b})$

Besides panoramic shots, an on-board camera bundle captures still images at preset increments on the road. The platform does not have the computational resources to take real-time images, but the storage itself could be used to compile a low-frame-rate video that gives you a visual summary of the exploration path. Because of the relatively small hardware overhead, this lightweight image acquisition strategy provides a very large benefit to situational awareness. These post-processed images are especially useful in search-and-rescue operations, which require manual inspection and coordination.



Figure 2. Example of photo merging during the AGV return to starting point, utilizing the periodically captured images as left (**a**), middle (**b**), and right (**c**), forming off-line the final panorama frame (**d**).

An external micro-SD card is used to store essential exploration statistics to facilitate data logging that may be kept at any point of decision. This consists of the dynamically constructed traversal graph, the decision-making checkpoints, energy use, and movement history. The data recorded is in the form of a tree and this enables backtracking and smooth continuation of missions without interruption.

The platform breaks the constraints of micro-controller-based systems by offloading the memory-intensive task of logging to a file-based long-term storage system to allow stateful and robust exploration over long periods.

3.4. Energy Awareness Subsystem

The system uses a rechargeable lithium-ion battery pack providing electrical energy to the micro-controller, the sensors, and the motor drivers. The battery level is continuously monitored by an integrated voltage sensor and comes back in real time to the control system. The energy-aware management layer uses this information in order to control exploration activities. Once the battery level has reached a preset level, the system takes action and does not extend the path beyond a specified level but starts backtracking. This protects against total loss of power and boosts the reliability of missions, especially when of longer duration or highly critical.

A special battery monitoring module regularly checks the supply of power. When the percentage of battery power is less than 50 percent, the autonomous ground vehicle (AGV) halts further exploration and launches a return-to-origin protocol through backtracking. Such a strategy helps to maintain an adequate amount of energy reserves that could provide a safe exit and leave the data integrity and the functions of the system intact.

During exploration, the AGV uses an extension technique of DFS and each of the vertices in the search tree to denote a decision point on a trip distance by using sensor-based distance measurements. Differing to the DFS logic, the nodes already traversed are kept in the form of a stack data structure, and in this way the AGV is able to backtrack in a rather

efficient manner whenever the exploration paths happen to become blocked or whenever the energy limit has been reached. The robot realigns itself in line so as to retrace through previous states and choose other routes.

The choice of the decision-making structure relies on classic algorithms of the graph search Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms adapted to run under limited memory. The AGV explores through its space in a tree-like manner, where each node will result in a visited location or a fork in the road. This traversal tree is recorded permanently to an external micro-SD card, making it possible to have strong backtracking and system recovery in case of interruptions.

The system programs traversal logic hand in hand with energy monitoring, achieving a dynamic equilibrium of the goals of exploration and the energy resources at hand. The method creates an embedded and low-weight solution that can assist normal Simultaneous Localization and Mapping (SLAM) systems, especially where platforms have limited resources or resources that are unpredictable in nature or energy-constrained. AN integrated view of the component deployment of the proposed system is illustrated in Figure 3.

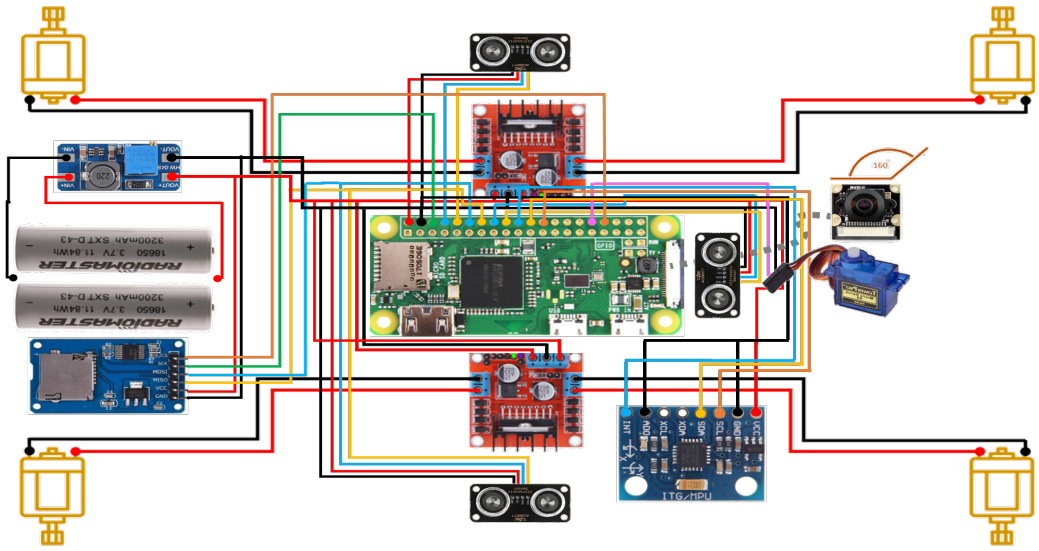


Figure 3. Component deployment.

4. Experimental Evaluation

4.1. Experimental Design

As a measure of estimating the performance of the various autonomous exploration algorithms in the maze environment, there arose a need to design and run sufficient simulations. This was aimed at evaluating and identifying the effectiveness and efficiency of three commonly used path-finding algorithms, namely Breadth-First Search (BFS), Depth-First Search (DFS), and A^*, in traversing and navigating through an unknown environment. In the experimental set-up, three major factors are distinguished:

- **Maze Configuration and Variability:** There were ten different maze configurations developed, which resulted in variability in layout and structural complexity. The mazes proposed were realistic in terms of creating an environment in which an automated guided vehicle (AGV) would find itself in a real-life situation, i.e., warehouses or structured indoor areas.
- **Starting Point Randomization**: Different starting points: Ten start points were chosen randomly for each maze so that there was a wide variety of initial conditions and no bias was caused by the particular position of entering the maze. This random-

ization was used to record an entire picture of the irregular algorithmic behavior in spatial variability.

- **Algorithm Deployment:** Using each of the ten starting positions in each of the mazes (that is, 100 total scenarios), the three algorithms, BFS, DFS, and A, were run separately. All the algorithms undertook a complete exploration phase beginning at the starting point, exploring the maze on the basis of their own logic and strategy.

In terms of the evaluation metrics, three main performance metrics were computed, which were averaged in all simulations:

1. **Percentage of visited cells**, denoted as C_v, describes the degree of exploration.
2. **Energy consumption**, denoted as E, is an estimate of the costs or resources that are consumed in the process of exploration.
3. **Efficiency rate**, denoted as R_e, depicts the ratio of the number of visited cells to the total energy consumed ($\frac{C_v}{E}$) and is taken as the rate at which the algorithms change energy into coverage of the area.

Regarding the repetition and averaging procedures deployed for the extraction of the results, the proposed design ensures that a number of 300 simulation runs was conducted (10 mazes $\times$10 start conditions $\times$3 algorithms). Each metric was then averaged overall trials to mitigate the effects of abnormalities and to produce reportable generalizations of the behaviors of the algorithms. In this systematic experimental set-up, the comparative analysis of the algorithms should have a high discriminating power, the experimental set-up should be randomized, and the comparison should be robust to enable the corresponding drawing of meaningful conclusions concerning the applicability of the algorithms to autonomous exploration tasks in structured environments.

On this point it should be noted that the entire research used a simulated environment in which there was control and repeatability in the performance of traversal algorithms in the same maze. This, however, comes with a catch, because simulation lacks vital real-world difficulties like wheel slip, sensor noise, and mechanical backlash. The current research project must therefore be taken as an experiment, of primary interest in assessing how energy-efficient and explorative BFS, DFS, and A^* are in situations of resource-constrained path planning. However, the system architecture was chosen such that it can be easily extended to physical implementation (i.e., using low-cost and readily available hardware components such as controllers, ultrasonic range finders, and wheel encoders). In future, practical tests on the hardware that validate the practical viability of the framework in an actual field environment will be performed and further robustness of the framework against physical uncertainties will be evaluated.

4.2. Software Implementation and Simulation Experiments

For the assessment of the proposed framework, a Python-based simulation framework has been developed using Python 3.13. This framework includes the following:

- Simulated hardware interfaces for motors, sensors, the camera, and storage.
- Modular code organization to separate control logic, hardware abstraction, and utility functions.
- A realistic approximation of sensor readings and battery behavior to validate the navigation algorithm.
- Logging features to trace decision-making and movement sequences.

The evaluation software developed for conducting the AGV maze exploration experiments [26] adopts a modular and layered architecture, thus possessing flexibility, scalability, and convenience in experiments. The system will employ an algorithm that automates the

use, application, recording, and assessment of three path-finding algorithms (BFS, DFS, and A^*) within a number of controlled simulations.

In Figure 4 an example of the software interface and an illustration of the routes followed are given. In Figure 4a the path followed by the AGV into an unknown maze is presented, while a split point is reached. Figure 4b shows the secondary path followed until the battery level falls below 50%, thus enabling the backtracking procedure. Figure 4c depicts the path followed through the backtracking procedure. As we can observe from Figure 4, the distance of backtracking is only half of the exploration distance, and the battery consumption should be approximately 25%. However, the assumption that backtracking requires exactly half the energy of exploration is idealized and does not account for direction changes and reduced sensor activity. On this point it is noteworthy that during the backtracking procedure the path is followed in reverse order, omitting the branches that do not include points in the path between the starting point and the current frontier point. That is, considering left and right sub-trees in the exploration path planning and that the AGV is currently in a leaf point of the right sub-tree (frontier), the backtracking procedure involves only the points that are included in the path from the starting point to the frontier that exist in the right sub-tree. Moreover, since the direction decisions are stored on the SD-card, the reverse order is followed by the AGV in order to return to the starting point without the utilization of the sensors for navigation and decision-making, thus resulting in a remaining battery level above 25%. Additionally, it is quite notable that a remaining battery level above 0% during the return of the AGV to the starting point could indeed be considered as inefficient (as this 25% could be utilized to extend the exploration frontier); however it is rather preferable to securely return to the starting point rather that starve from energy in a non-accessible point and require human intervention to retrieve the AGV.

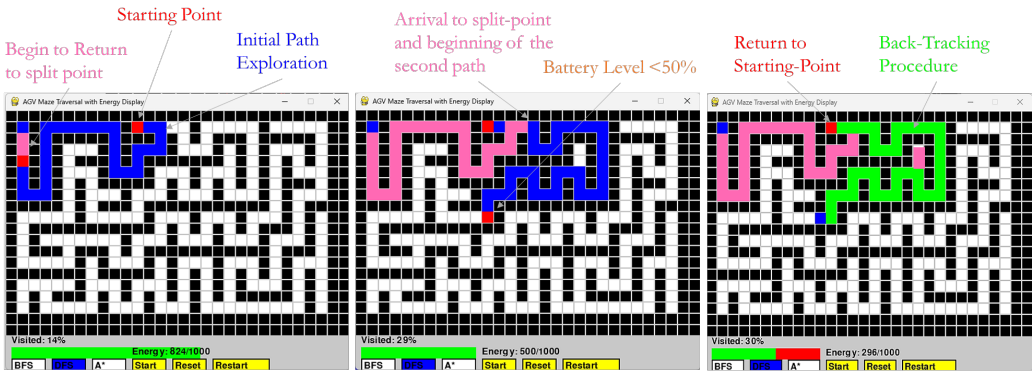


Figure 4. Simulation application interface depicting the initial path exploration and the return of the AGV to the first split point (**a**), the exploration procedure until the battery level falls to 50% (**b**), and the deployment of the backtracking procedure to the starting point (**c**).

The key modules that are included in the software are the following ones, Maze Manager, Start Point Generator, Algorithm Engine (BFS, DFS, or A^*), Simulation Controller, Metrics Collector, Results Aggregator, and the Data Logger/ Exporter, where each module communicates through a clearly specified interface to ensure that each is reusable and maintainable. The following procedures depict how the system carries out the experiments:

1. **Maze Initialization:** The Maze Manager loads or generates 10 distinct maze configurations. Each maze is represented as a 2D grid or graph structure, where nodes represent cells and edges denote valid movements.

2. **Starting Point Selection:** For each maze, the *Start Point Generator* randomly selects 10 valid starting positions (i.e., non-wall cells). These starting positions are stored to ensure consistency across algorithm evaluations.
3. **Algorithm Deployment Loop:** The simulation software generates 10 random mazes and randomly selects 10 valid starting positions within the maze. For every starting point, each of the three exploration algorithms (BFS, DFS, and A^*) is executed independently. During the exploration, the system tracks the number of visited cells, estimates energy consumption based on movement or action steps, and computes the efficiency as the ratio of visited cells to energy used.
4. **Results Aggregation:** After completing all 300 simulations (10 mazes $\times$ 10 points $\times$ 3 algorithms), the *Results Aggregator* computes the values of C_v, E, and R_e

On this point it is quite notable that the above-discussed experiments were not performed in the environments of static mazes where roads and barriers could not be changed during exploration. This design decision aims to isolate the relative performance of traversal algorithms under constrained sensing budgets and constrained energy budgets, without the added variability of dynamic or physics-based simulations. Although this minimalist model offers transparency in the analysis of the trade-offs in algorithms, the model fails to address the complete set of issues in the dynamic environment that requires moving obstacles, sensor noise, or actual disaster situations. In the current study, we have not carried over our tests to other platforms, like Gazebo or Webots; however, since the proposed framework is modular and extensible, integration with simulation platforms like Gazebo or Webots, or real-life post-disaster datasets, is a logical next step to the current work.

Regarding the simulation environment, and particularly the maze size and complexity levels, in order to create interconnected but dynamic pathways, the maze took the form of a 30 $\times$ 20 grid in which a randomized maze generation algorithm was used to create the connected maze. There were ten distinct mazes constructed, with different aspects of topological complexity overall. The initial states of every maze were mixed up, and there were 10 random start locations in the free cells that were reachable. An initial 1000-unit energy level was assumed, from which 1 unit was subtracted per move, while 2 units were subtracted when the sonar was being utilized (each time the direction was changed), and 3 units were subtracted for each servo driven-camera rotation (each step of movement). The search was conducted until the rest of the energy dropped to 50 % or less of the capacity or the AGV returned to the starting point by initiating a backtracking procedure. Frontiers that were observed are algorithm-specific, while in order to provide a fair comparison, the same maze and initial starting points were used when testing each of the three algorithms. Measures of performance focused mostly on how many of the available cells were reached prior to the terminal, as well as the remaining energy at end of the backtracking procedure, an indication of the movement efficiency and cost of safe escape. Based on the same environmental constraints, where variability was still experienced because of the randomized maze structures and starting points, the given experimental set-up provided controlled comparison of the three exploration algorithms.

4.3. Discussion of the Exhibited Results

Figure 5 gives a summary of the performance of the three traversal algorithms (BFS, DFS, and A^*) with respect to the percentage of visited points and energy consumed in the process of traversal. Performance indicators have the mean and standard deviation as well as 95% confidence intervals (CIs) of the two variables. In general, the results point out a trade-off between energy consumption and node exploration. BFS exhibited low node visiting, high energy consumption, and medium variability. It is applicable to applications that need comprehensive coverage with predictable search modes. On the

other hand, DFS provided great visitation of the nodes, low mean energy, and higher variability in energy. It is best suited to applications where energy proliferation is favored over exploration uniformity. Finally, A^* balances the two parameters of node visiting and energy. The potential of heuristic-guided search is lower unnecessary search, but this comes with energy overhead. It delivers reasonable performance with low variation. Such results imply that such choices of the algorithm have to be directed to the specific purpose of the traversing: energy minimization, the thoroughness of the search, or a balance between the two. The reversing tendencies between visiting nodes and energy consumption emphasize the necessity to implement several performance indicators in the performance of algorithms.

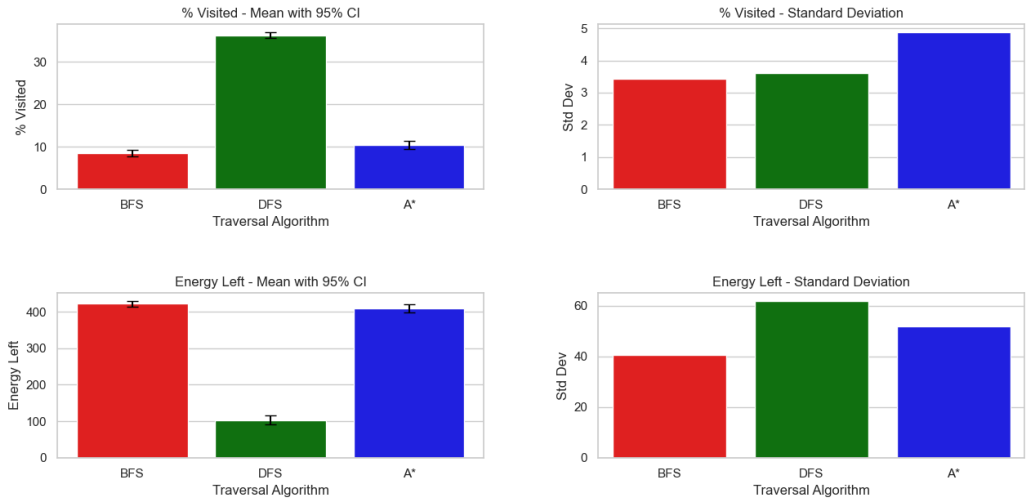


Figure 5. Standard deviations and confidence intervals for the performance indicators of visited cells' percentage and energy consumption exhibited by the three exploration algorithms.

Regarding the percentage of visited points, DFS exhibits the highest mean nodes in a visit (36.25%), which is much higher than BFS (8.49%) and A^* (10.38%). This is in line with the nature of DFS being much more node traversing-heavy along a path and then coming back up, which means that it frequently traverses more of the search space complex than BFS and other algorithm methods, even heuristic-led ones. BFS maximized the number of nodes explored, with a mean of 8.49% nodes explored as compared to the others, as it has level-order exploration, meaning that exploration avoids useless deep search. A offered an intermediate mean of 10.38%, showing evidence of heuristic bulwarks in cutting off the number of measured nodes when compared to DFS but with a marginally higher heuristic evaluation overhead than BFS. These observations are sustained by the standard deviations and confidence intervals. DFS results indicated that there was a tight spread (Std = 3.61), showing surface stability between trials. BFS and A^* displayed an almost equally consistent variation (Std = 3.43 and 4.88, respectively) due to slight deviations in node visitation perhaps caused by certain graph topologies as well as the accuracy of heuristics.

Concerning energy consumption, DFS had the lowest mean energy consumption (103.22 units) compared to BFS (422.23 units) and A^* (409.17 units). It might seem that this outcome is counterintuitive, since DFS is able to traverse more nodes, yet the metric used to measure energy probably notes the efficiency of following a path or the cost of traversing it, not merely the amount of nodes traversed. DFS could have discovered shorter paths or paths requiring less energy in the deeper levels of the search space, whereas BFS and A^* used up energy on traversing several nodes in the shallower section or checking heuristic values. BFS and A^* consumed about the same level of energy, with BFS a little over the

average. The standard deviations show that the DFS energy consumption was distributed over a wider range (Std = 61.99) as compared to BFS (Std = 40.67) and A^* (Std = 51.88), implying that although DFS may result in low-energy paths, it is less predictable over trials or various graph experiments. The differences in energy consumed by the algorithms are confirmed to be statistically significant, and confidence intervals confirm that there is little overlap between DFS and the rest of the methods.

The results exhibited over a series of repetitive simulation experiments investigating the exploring capabilities of the three algorithms (BFS, DFS, and A^*) are capable of offering a set of valuable details concerningthe effectiveness and efficiency of the considered techniques applied to searching mazes within different starting points, as shown in Figures 6–8. The performance of the three exploration approaches was evaluated over 300 experiments, comprising 10 different mazes and 10 different starting points per maze. The results were averaged across all runs and analyzed based on three primary metrics: the percentage of visited cells, energy consumption, and efficiency rate.

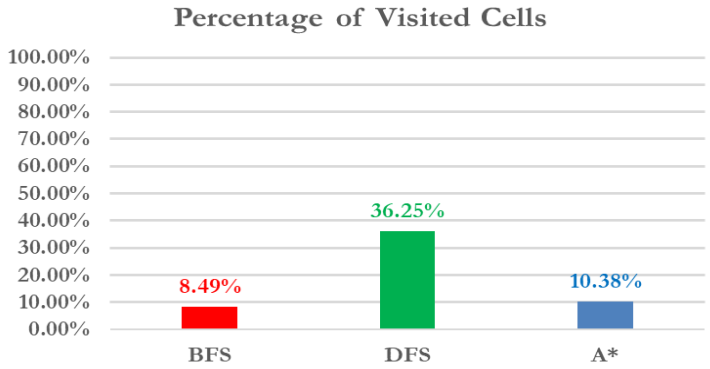


Figure 6. Averaged percentage of visited cells over a series of simulation experiments utilizing BFS, DFS, and A^* for different maze shapes and starting points.

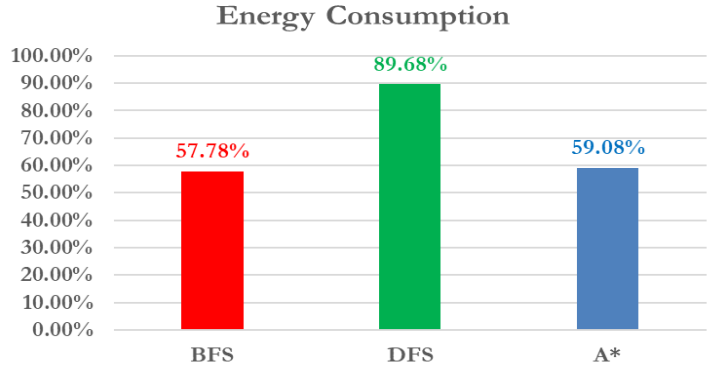


Figure 7. Averaged percentage of energy consumption over a series of simulation experiments utilizing BFS, DFS, and A^* for different maze shapes and starting points.

Coverage of the exploration (i.e., the percentage of visited points) is a quantitative indicator of how thoroughly each of the algorithms explores the maze. Concerning the results depicted in Figure 6, the visited cell percent shows the size of the explored area that was covered by the algorithm before it reached the target. DFS had a higher exploration rate, with an average of 36.25% of the space in the maze being visited. This is indicative of the fact that it does not backtrack until it has to and in turn explores too much. On the contrary, the visitation rate of BFS was the lowest at 8.49 percent on account of its breadth-first approach to traversing the layers and tendency to locate the shorter routes sooner. A^*

kept the visitation rate low (10.38%), and the advantage of heuristic guidance ensured that it reduced the search space and emphasized the best directions.

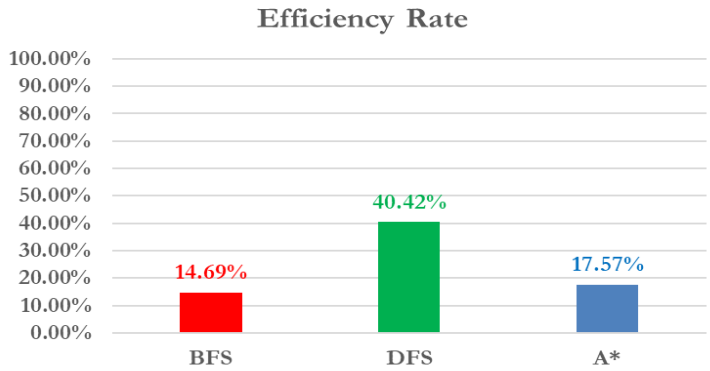


Figure 8. Averaged efficiency rates over a series of simulation experiments utilizing BFS, DFS, and A^* for different maze shapes and starting points.

Energy consumption is an indicator (cost) of the algorithm operation (e.g., steps or computational effort). Regarding the results depicted in Figure 7, due to its exhaustive exploration, energy consumption was highest for DFS at 89.68%. The A^* performance of 59.08% was a trade-off between informed search and computer work. BFS was the least energy-intensive at 57.78%; its significant energy efficiency might be due to its potential to execute a vast number of nodes before reaching the target, particularly in networks that have low connectivity.

The efficiency rate gives an aggregate idea of the relative effectiveness of each algorithm in translating exploration and energy into successful path-finding. Finally, observing the results depicted in Figure 8, DFS proved to be the most efficient one with a 40.42% rate, indicating that even though it used a lot of energy and searched vast fields, it was able to locate routes that were quite close to the destination in most cases. A^* reached a middle rate of 17.57% because of its intermediate trade-off between path quality and the cost of calculations through the heuristic-based navigation. BFS was also the least efficient and had a rate of 14.69%, implying that its exhaustive search algorithm, though procedural, can lead to poor or unnecessary time-consuming routes.

Generally, the exhibited results illustrated in Figures 6–8 depict the findings obtained at the end of different simulation experiments, which illustrate some of the major trade-offs among exploration breadth, energy cost, and overall performance. DFS, despite being energy-intensive, is the most comprehensive in its coverage and has a high efficiency ratio. A^*, in turn, is less grainy than DFS and has a decent ratio between energy costs and efficiency, which is why it can be recommended for applications in the case of goal-targeted exploration. BFS has the highest power conservation, but it is less than effective in its coverage. In real-world practical AGV use and in applications that are energy-critical, perhaps the better compromise between energy conservation and maze coverage would be A^*. DFS will be handy where exhaustive exploration is necessary and BFS where exploration is limited in low-risk environments.

5. Conclusions

In this paper we presented a low-cost, energy-aware exploration framework for autonomous ground vehicles in hazardous environments. This system sets out to demonstrate how potentially effective exploratory events can be proficiently achieved with reduced computational and energy assets by making use of traversal algorithms (BFS, DFS, and A^*) within the framework of the environment of a lightweight micro-controller-based platform.

The developed AGV system incorporates sonar-based navigation, live energy reporting, an SD card-type logging system to facilitate safe exploration, and methodical backtracking during a limited supply of energy. An experimental simulation where 300 runs were run on different mazes and starting points showed that DFS outperformed the other methods in both exploration coverage and efficiency, whereas A^* provided excellent interchange in energy consumption and coverage. Energy-conservative BFS is still suboptimal within the features of exploratory completeness and efficiency. These results provide strong evidence of the feasibility of making classical algorithms energy-aware in unknown terrains.

5.1. Potential and Limitations

The proposed framework provides solid energy-aware unmanned exploration. The system's architecture with a low hardware footprint and energy-aware control logic makes it especially applicable in environments where SLAM-based or vision-centric solutions are less efficient. In addition, the organized image logging is capable of allowing off-line visual reconstruction without exhausting the real-time processing, thereby allowing post-mission analysis of situations that are safety-critical.

Nevertheless, the framework is limited too. The existing application is also based on rather primitive sensors, which constrains the accuracy of the environment perception and identification of obstacles. The traversal algorithms are computationally efficient; however they are not adaptive and learning-based and thus might not best deal with dynamic terrains or unpredicted terrain in comparison with more recent AI-based techniques. Last but not least, off-line processing of visual stimuli constrains instant communication in exploration expeditions.

One shortcoming of this current prototype is that it assumes precise (and discrete) 90-degree rotations without real-time feedback of sensor heading. Although this simplification results in ease of implementation and a measured evaluation of algorithmic performance over a controlled environment, it fails to encapsulate mechanical drift, wheel slippage, and motor inaccuracies, which are typical of real-world robotic platforms. Smaller angular errors may stack up over consecutive turns, causing directional mapping errors or distortion, inaccurate path reconstruction, or total navigation failure. Future versions should add significant robustness by using low-cost inertial sensing (e.g., MPU6050) to provide angular velocity and orientation feedback, allowing adjustments to be made to the calculated heading in real time to eliminate drift error. Alternatively, wheel encoder information or the turn time heuristics could be used to detect and correct systematic rotational errors without adding much in hardware expense. These mechanisms would enable the robot to keep a constant orientation in a longer-lasting exploration process, decreasing the risk of accumulated errors and increasing the finality of such map-creating and route-retracing processes.

Additionally, another limitation of the current research is that the experimental assessments were conducted solely in a simulation set-up. The selection of the simulation was driven by the fact that this method can offer a controlled, repeatable environment to complete systematized testing of traversal algorithms on a standard example of a maze, which in turn will guarantee equitable comparisons and consistent measurements of performance metrics (e.g., energy consumption and exploration efficiency). However, such dependence on simulation always limits the generalizability of the findings because it does not reflect the real-world uncertainty that would be encountered in real deployment. Wheel slip, surface irregularity, sensor noise, mechanical backlash, and environmental variation can all have a strong impact on the accuracy of navigation and system robustness. Even though the framework proposed in this work has not yet incorporated preliminary hardware verification, it mainly focuses on the development of a prototype platform. Such

activities will allow systematic experimental verification in real-world conditions, such as sensor reliability, actuator dynamics, and energy efficiency in uncontrolled environments. Future research should focus on applying these results to non-simplified, field-deployable autonomous ground vehicle systems. Nevertheless, the framework offers a lightweight and energy-conscious base that could inform the design of pragmatic, low-cost exploration platforms in dangerous and resource-limited environments.

Moreover, although this study introduces the framework as a complement to SLAM, it does not replicate all SLAM core functionalities, including loop closures, probabilistic localization, and metric mapping. Instead, the strategy avoids the overheads of global metric maps, instead appealing to corresponding lightweight, direction-based node transitions. This is by design; that is, the focus is on energy-frugal exploration of topologically structured or bounded environments (i.e., arenas, warehouses, mazes, or generic indoor grids), where the computational and energy demands of traditional SLAM are excessively high. It is, however, noted that this restricts use in larger or more dynamic applications, where wide-region consistency of maps and probabilistic localization is needed. Future research will consider how a hybrid scheme can be used to leverage the simplicity and low cost of our framework with coarse distance estimation or minimal localization feedback, thus further broadening the applicability of operations with efficiency advantages.

It should be stressed, nevertheless, that the main goal of this study was to present an algorithmic benchmarking framework under controlled conditions. Separating the navigation algorithms (BFS, DFS, and A^*) in hardware-agnostic terms, we can develop a reference point for the performance of these algorithms and their trade-offs. Such a starting point is essential since it enables follow-up work to rule out contributions of the algorithm that are instead due to mechanical or sensing constraints. However, we take into consideration that the step toward actual implementation involves building some of these sensing and correction mechanisms into it, and our updated ideas point to them as the obvious and practical vectors of a further enhancement.

5.2. Future Research

An interesting aspect regarding the future directions of this work is to analyze the proposed system within a dynamic and physics-based environment. This can also support richer sensor uncertainties, stronger robot dynamics, and dynamic obstacles on more realistic simulation platforms, e.g., Gazebo or Webots, making testbeds closer to real-world exploration challenges. Also, benchmarking with reality-based post-disaster datasets (for example, collapsed buildings, fire-damaged interiors, or biohazard zones) would allow the system to perform under real-life and unexpected scenarios and reinforce the usability of the system in mission-critical applications. In the present work, the lack of hardware as well as the available resources necessitated the constraint of the static environments, but the modularity and extensibility of the platform will help to couple the platform core with more powerful simulation environments and real-world data in subsequent extensions.

Additionally, to fill the gap of real-world data and deployment in real scenarios, a proper road map of further deployment of the hardware has been stipulated. In particular, we intend to integrate the proposed framework into our Raspberry Pi Zero W platform, the low-power processing capacity and wireless connectivity of which will be used. The navigation subsystem will be enriched with commercially available sensors, such as low-cost ultrasonic rangefinders to sense obstacles, wheel encoders to measure distance covered, and additionally an optional IMU module, which will correct the heading. Such a software/hardware setting will enable us to replicate the simulation settings' physical requirements and evaluate the robustness of the system to drift, slip, and sensor noise in the experimental conditions. Moreover, actual experiments will allow the assessment of

the failure recovery aspect of runtime and energy-efficient search, as well as the efficiency of traversal under realistic conditions. This roadmap makes the shift from the simulation to hardware possible and does not contradict the low-cost and resource-constrained nature of the proposed system.

Building upon the established foundation, several promising directions for future research emerge. First, integration of machine learning or reinforcement learning techniques could enhance the adaptability of the exploration strategy in dynamic environments. Second, the energy management system can be refined through predictive models that estimate remaining exploration potential before initiating return-to-base protocols. Third, expanding the visual system to include lightweight SLAM capabilities or real-time low-resolution mapping could improve immediate situational awareness. Additionally, multi-agent coordination strategies could be explored to allow cooperative exploration among multiple AGVs. Finally, real-world deployment and testing in controlled post-disaster scenarios or search-and-rescue mock-ups would provide valuable insights into the robustness and practical constraints of the system under field conditions.

Author Contributions: Conceptualization, I.P.; methodology, I.P., M.N.A., I.V., and M.A.; validation, I.V.; writing—original draft preparation, I.P., M.N.A.; writing—review and editing, I.P., M.N.A. and M.A.; supervision, M.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Acknowledgments: We would like to express our sincere gratitude to Christos Sarantidis, undergraduate student and valued member of our team, for his significant contribution in conducting the experiments of the evaluation of our framework.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Maidana, R.; Granada, R.; Jurak, D.; Magnaguagno, M.C.; Meneguzzi, F.R.; de Morais, Amory, A. Energy-aware path planning for autonomous mobile robot navigation. In Proceedings of the 33rd International Florida Artificial Intelligence Conference, North Miami Beach, FL, USA, 17–20 May 2020.
2. Rappaport, M. Energy-aware mobile robot exploration with adaptive decision thresholds. In Proceedings of the ISR 2016: 47st International Symposium on Robotics, Munich, Germany, 21–22 June 2016; VDE: Offenbach am Main, Germany, 2016; pp. 1–8.
3. Mei, Y.; Lu, Y.H.; Lee, C.G.; Hu, Y.C. Energy-efficient mobile robot exploration. In Proceedings the 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006, Orlando, FL, USA, 15–19 May 2006; IEEE: Piscataway, NJ, USA, 2006; pp. 505–511.
4. Teixeira, J.M.X.; Pimentel, N.; Barbier, E.; Bernard, E.; Teichrieb, V.; Chaves, G. Low-Cost 3D Reconstruction of Caves. In Proceedings of the 18th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Application VISIGRAPP (5: VISAPP), Lisbon, Portugal, 19–21 February 2023; pp. 1007–1014.
5. Rejab, K.S.; Abd-Al Hussain, S.M.N. Design and implementation of intelligent mobile robot based on microcontroller by using three ultrasonic sensors. *Int. J. Curr. Eng. Technol.* **2017**, *7*, 2051–2056.
6. Azeta, J.; Bolu, C.; Hinvi, D.; Abioye, A.A. Obstacle detection using ultrasonic sensor for a mobile robot. *Iop Conf. Ser. Mater. Sci. Eng.* **2019**, *707*, 012012. [CrossRef]
7. Kerstens, R.; Laurijssen, D.; Steckel, J. ERTIS: A fully embedded real time 3d imaging sonar sensor for robotic applications. In Proceedings of the 2019 International Conference on Robotics and Automation (ICRA), Montreal, QC, Canada, 20–24 May 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 1438–1443.
8. Meneses, J.; Barrientos, A.; Gonzalez, J. Using Optical Flow for Robot Navigation in Low-Power Platforms. *Sensors* **2016**, *16*, 2122. [CrossRef]
9. Khan, S.; Daudpoto, J. Development of Low-Cost Autonomous Robot. In Proceedings of the 2019 Innovations in Intelligent Systems and Applications Conference (ASYU), Izmir, Turkey, 31 October–2 November 2019; pp. 1–6. [CrossRef]

10. Ziouzios, D.; Kontarinis, G. Raspberry Pi Based Obstacle Avoidance Robot Using Ultrasonic Sensors. *J. Eng. Sci. Technol. Rev.* **2019**, *12*, 145–150. [CrossRef]
11. NTNU Open. *Obstacle Avoidance Robot with Ultrasonic Sensors Using Raspberry Pi*; NTNU Open: Trondheim, Norway, 2021; NTNU Project Archive.
12. Seewald, A. ; Lerch, C. J.; Chancán, M.; Dollar, A. M.; Abraham, I. Energy-Aware Ergodic Search: Continuous Exploration for Multi-Agent Systems with Battery Constraints, 2024 IEEE International Conference on Robotics and Automation (ICRA), Yokohama, Japan, 2024; pp. 7048–7054. [CrossRef]
13. Ben Naveed, A.; Jain, A.; Lavelle, M. ECLARES: Energy-Constrained Ergodic Control for Exploration and Environmental Sensing. *Robotics* **2021**, *10*, 115. [CrossRef]
14. Zhang, Y.; Ai, Z.; Chen, J.; You, T.; Du, C.; Deng, L. Energy-Saving Optimization and Control of Autonomous Electric Vehicles With Considering Multiconstraints. *IEEE Trans. Cybern.* **2022**, *52*, 10869–10881. [CrossRef] [PubMed]
15. Dovgan, E.; Jogan, M. Energy-Aware Navigation with Dynamic Weighting for AGVs. *IFAC-PapersOnLine* **2020**, *53*, 8124–8129. [CrossRef]
16. El-Hussieny, H.; El-Sayed, A. Modified DFS Algorithm for Robotic Path Planning in Unknown Terrains. *Int. J. Comput. Appl.* **2018**, *179*, 1–6. [CrossRef]
17. Parasuraman, S. Sensor Fusion for Mobile Robot Navigation: Fuzzy Associative Memory. *Procedia Eng.* **2012**, *41*, 251–256, ISSN 1877-7058. [CrossRef]
18. Russell, S.; Norvig, P. *Artificial Intelligence: A Modern Approach*, 2nd ed.; Pearson Education: London, UK, 2003.
19. Lambers, M.; Schindler, S. Visual Odometry Using Offline Image Logs in UAV Systems. *Sensors* **2019**, *19*, 910. [CrossRef]
20. Rodríguez-Martínez, E.A.; Flores-Fuentes, W.; Achakir, F.; Sergiyenko, O.; Murrieta-Rico, F.N. Vision-Based Navigation and Perception for Autonomous Robots: Sensors, SLAM, Control Strategies, and Cross-Domain Applications—A Review. *Eng* **2025**, *6*, 153. [CrossRef]
21. Zhou, T.; Wang, Y. Reconstructing Visual Environments from Sparse Image Logs. In Proceedings of the International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISAPP), Madeira, Portugal, 27–29 January 2018; pp. 623–630.
22. Reyes, R.; Tovar, B. Energy Prediction in Mobile Robots Using Artificial Neural Networks. *Sensors* **2020**, *20*, 762. [CrossRef]
23. Byun H, Yang S. An energy-efficient and self-triggered control method for robot swarm networking systems. *Meas. Control* **2022**, *55*, 898–907. [CrossRef]
24. Lai, J.; Wu, Z.; Ren, Z.; Tan, Q.; Xiao, H. Optimal navigation of an automatic guided vehicle with obstacle constraints: A broad learning-based approach. *IEEE Trans. Emerg. Top. Comput. Intell.* **2025**, *9*, 3010–3024. [CrossRef]
25. Nowakowski, M.; Kurylo, J.; Braun, J.; Berger, G.S.; Mendes, J.; Lima, J. Using LiDAR data as image for AI to recognize objects in the mobile robot operational environment. In Proceedings of the International Conference on Optimization, Learning Algorithms and Applications, Ponta Delgada, Portugal, 27–29 September 2023; Springer Nature: Cham, Switzerland, 2023; pp. 118–131.
26. Project on GitHub. Available online: https://github.com/IosifPolenakis/EA-AGV/blob/main/maze.zip (accessed on 11 September 2025).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.